1. Loss: how wrong is one guess?

label y (1 = corner, 0 = not), guess p (0 to 1)

$$L = -\big[\, y \ln p + (1 - y) \ln(1 - p) \,\big]$$

y = 1, p = 0.99 → $L = -\ln 0.99 \approx 0.01$

y = 1, p = 0.01 → $L = -\ln 0.01 \approx 4.6$

2. Step: change the weights w a little

learning rate η = how big the step is

$$w \;\leftarrow\; w - \eta \nabla_w L$$

Learning from answers

Loss

One number for how wrong a guess is. Confident and wrong costs a lot.

Gradient step

Weights = the network's adjustable numbers. ∇L points where the loss grows, so step the other way.

In Swift, a U-Net (a network that answers for every pixel) marks the 4 gate corners.

Sources: Nature, 2023 (Kaufmann et al.); AlphaPilot, arXiv 2005.12813

Reusing what is learned

Start trained

Copy the weights of a network that already works, then keep training on new data

Small steps

A small step size η keeps the weights near where they started

Swift

10^8 steps in simulation, then 2×10^7 steps with real flight data

After T steps of size η, with gradients of size at most G, the weights moved at most η·T·G. Too big a step erases what the network knew.

Sources: Nature, 2023 (Kaufmann et al.); Yosinski et al., arXiv 1411.1792; EWC, arXiv 1612.00796; LoRA, arXiv 2106.09685

Learning from a score

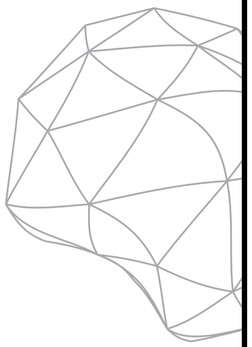
Return

All future rewards added up. Near rewards count more because γ < 1.

Policy gradient

Policy = the network that picks actions. Make actions with a positive advantage A likelier.

Swift's reward: progress to the next gate and the gate in view, minus jerky commands and crashes.



1. Return: all future rewards, today's count most

reward r after each step, discount γ between 0 and 1

$$G = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots$$

rewards 1, 1, 1 and γ = 0.9 → $G = 1 + 0.9 + 0.81 = 2.71$

2. Policy gradient: make good actions likelier

policy π with weights θ, advantage A = how much better than average

$$\nabla J(\theta) = \mathbb{E}\left[\nabla_\theta \ln \pi_\theta(a \mid s) \cdot A \right]$$



Sources: Nature, 2023 (Kaufmann et al.); PPO, arXiv 1707.06347; GAE, arXiv 1506.02438

PPO: small, safe updates

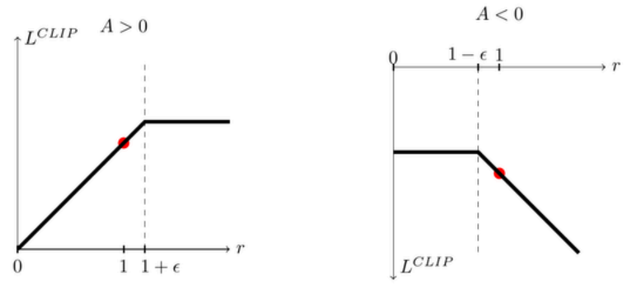
PPO stops one update from changing the policy too much. A is the advantage: how much better than average the action was.

Ratio: how much did the policy change?

$$r = \frac{\pi_{\text{new}}(a \mid s)}{\pi_{\text{old}}(a \mid s)}$$

Clipped objective, $\epsilon = 0.2$

$$L = \mathbb{E}\left[\min\left(rA,\ \text{clip}(r,\ 1-\epsilon,\ 1+\epsilon)\,A\right)\right]$$



Ratio r

New policy over old policy for the same action. r = 1 means no change

Clip

Gains from r outside 1 ± ε are cut off. Here ε = 0.2

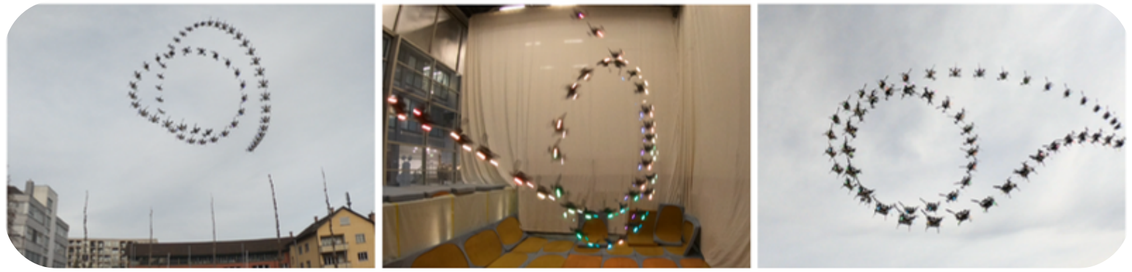
Why

One bad step cannot wreck what the policy has learned

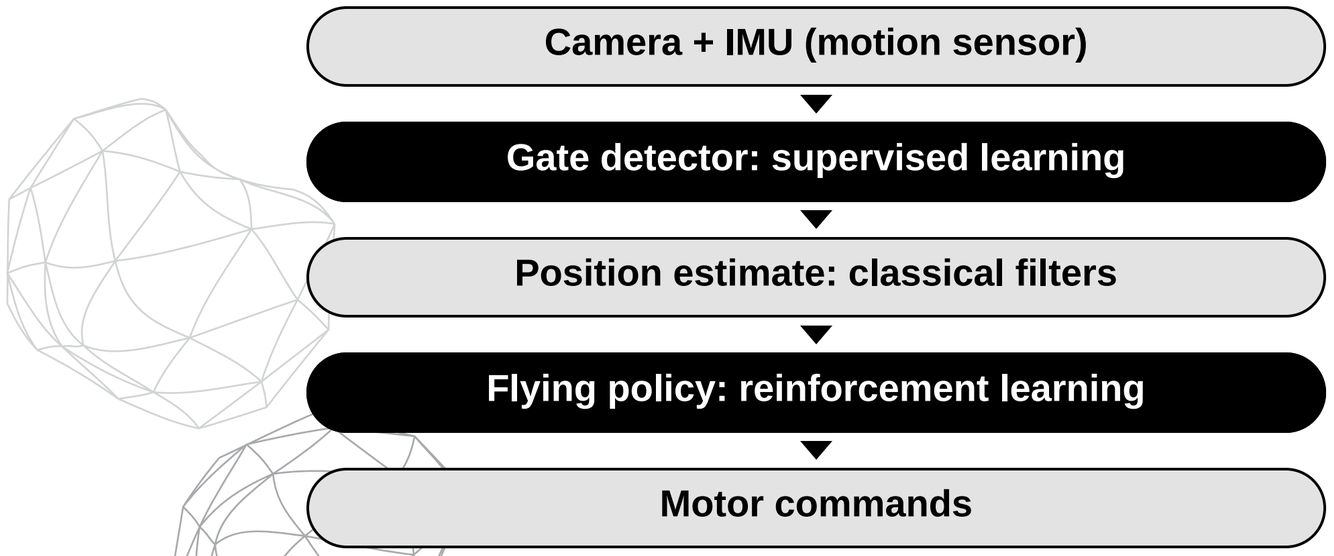
Sources: PPO, arXiv 1707.06347 (Fig. 1 and Eq. 7); Nature, 2023 (Kaufmann et al.)

Learning by copying

A student copies a teacher that sees the exact state (only in simulation). DAgger: the teacher labels where the student flew.



Barrel roll, power loop, Matty flip: trained only in simulation



Black: learned. Gray: sensors and classical methods.

Invent or copy?

Should a machine invent its own strategy, or copy a human?

Sources: Nature, 2023 (Kaufmann et al.); Deep Drone Acrobatics, RSS 2020 (arXiv 2006.05768); DAgger (arXiv 1011.0686)